

Microservicios, Persistencia NoSQL y Atributos de Calidad

CG

May 18, 2026

1. En la arquitectura de FinCore, un desarrollador propone implementar el patrón **Microservice Chassis**. cual es el principal beneficio arquitectonico que se busca con esta decisión?

Reducir el tiempo de comercialización y asegurar la consistencia transversal al **encapsular** como logs, seguridad y métricas en una plantilla reutilizable.

2. al aplicar el patrón **Externalized configuration**, que impacto se genera directamente sobre los ASRs del sistema?

Favorece la modificabilidad al permitir cambios de parámetros en tiempo de ejecución o arranque sin recompilar el artefacto, pero puede añadir un riesgo de seguridad si no se cifran los almacenes centralizados.

3. un Message Broker con garantía de entrega 'At least once' duplica un mensaje debido a un fallo de confirmación de red. Si el consumidor implementa el patrón Idempotent Consumer utilizando Redis como almacén de IDs distribuidos, ¿qué acción toma el sistema?

Verifica si el ID de mensaje ya existe en Redis; al encontrarlo, descarta el mensaje duplicado e impide la **re-ejecución** de la lógica de negocio.

4. Considera la estrategia de despliegue 'Multiple Service Instances per Host'. ¿Cuál es su principal debilidad en un entorno de alta concurrencia como el de BITE.co?

La falta de aislamiento de recursos, lo que permite que una sola instancia sature la CPU o memoria del host, degradando la disponibilidad de todos los demás servicios compartidos. *compartir el mismo espacio de sistema operativo significa que no hay limites estrictos de hardware por proceso, induciendo contención de recursos*

5. En la estrategia de despliegue 'Service Instance per VM', ¿cuál es el trade-off directo en comparación con el uso de contenedores?

Se maximiza el aislamiento de fallas y seguridad a nivel de hipervisor, pero a costa de un mayor consumo de recursos y tiempos de aprovisionamiento elevados. *cada VM arranca un SO completo, garantizando fronteras rígidas de hardware y red, pero penalizando el tamaño del artefacto y su velocidad de despliegue*

6. El documento del Grupo 1 describe la estrategia 'Service Instance per Container'. ¿Por qué la seguridad se evalúa con una calificación media/baja (B) en su matriz de trade-offs?

porque los contenedores comparten el mismo kernel del sistema operativo del host, lo que amplía la superficie de ataque si un proceso logra romper el aislamiento del espacio de nombres(namespace). *a diferencia de las VMs con kernels separados, una vulnerabilidad a nivel de kernel en el host puede comprometer potencialmente a todos los contenedores residentes*

7. Bajo un esquema de 'Serverless Deployment' (ej. AWS Lambda), ¿cuál de las siguientes situaciones describe el fenómeno conocido como 'Cold Start'?

La latencia adicional que ocurre cuando llega una petición y la plataforma debe aprovisionar un contenedor efímero, descargar el código e iniciar el entorno de ejecución desde cero. *al no haber servidores encendidos continuamente, la primera llamada sufre un retraso mientras la plataforma Serverless orquesta la infraestructura base*

8. En el patrón de descubrimiento 'Client-side Discovery', ¿cuál es el rol exacto del cliente que desea invocar a otro servicio?

Consultar activamente al service registry las ubicaciones de red disponibles, aplicar localmente un algoritmo de balanceo de carga y direccionar la petición directamente a la instancia seleccionada. *en el enfoque de cliente, la inteligencia de consulta de catálogo y la distribución del tráfico reside en el proceso que origina la llamada*

9. En un esquema de 'Server-side Discovery', ¿qué atributo de calidad se ve amenazado al introducir un enrutador intermedio (como un Balanceador de Carga)?

La disponibilidad, puesto que el balanceador de carga centralizado se convierte en un punto único de falla, si no se configura con redun-

dancia activa. *toda comunicación obligatoriamente pasa por este nodo intermedio, si colapsa, la comunicación interservicios se interrumpe por completo*

10. El equipo de arquitectura debate entre 'Self Registration' y '3rd Party Registration'. ¿En qué escenario es técnicamente indispensable optar por '3rd Party Registration'?

cuando se cuenta con un entorno poliglota complejo y servicios legados de caja negra cuyo código fuente no puede modificarse para embeber lógica de infraestructura de red.

11. En el patrón 'API Composition', si el compositor orquesta llamadas síncronas en paralelo a tres servicios independientes para armar una vista unificada, ¿cómo se calcula teóricamente el tiempo mínimo de respuesta total de la consulta?

es igual al tiempo de respuesta del microservicio que responda más lento, más la sobrecarga de red y el procesamiento de agregación del propio compositor.

12. La segregación de responsabilidad de comandos y consultas (CQRS) resuelve la contención de recursos en alta concurrencia. Sin embargo, ¿qué costo crítico impone sobre el sistema?

introduce consistencia eventual, lo que significa que las consultas pueden retornar datos transaccionales desactualizados por milisegundos mientras se propaga la sincronización asíncrona.

13. En el patrón 'Event Sourcing', ¿cómo se determina el estado actual de una entidad de negocio en un momento dado?

Recuperando la secuencia cronológica completa de eventos mutativos inmutables guardados en el Event Store reproduciendolos secuencialmente en memoria (rehidratación)

14. Para mitigar la degradación de desempeño que causa reproducir miles de eventos históricos en Event Sourcing, ¿qué mecanismo técnico se debe incorporar?

La generación periódica de instantáneas (snapshots) que guardan el estado consolidado cada cierto número de eventos, permitiendo rehidratar solo los eventos ocurridos después del último Snapshot.

15. De acuerdo con el documento de diseño detallado NoSQL, ¿cuál es la premisa fundamental para modelar datos en bases documentales?

Diseñar las estructuras de los documentos adaptandose estrictamente a los patrones de acceso y consultas de la aplicación, aceptando la desnormalización si favorece el desempeño.

16. En una colección de productos de comercio electrónico con propiedades altamente heterogéneas, se aplica el 'Attribute Pattern'. ¿Cuál es el cambio estructural que sufre el documento JSON?

se encapsulan las propiedades variables dentro de un arreglo único de subdocumentos con la estructura fija llave-valor [*"k": "nombre-propiedad", "v": "valor"*]

17. Al utilizar el 'Attribute Pattern' en MongoDB, ¿cuál es el beneficio directo sobre la infraestructura de indexación?

Habilita la creación de un único índice compuesto sobre las propiedades del arreglo, evitando la explosión de índices individuales pesados.

18. El patrón NoSQL 'Schema Versioning' dictamina que los documentos antiguos y nuevos coexistan en la misma colección de producción. ¿Cómo maneja la aplicación la disparidad de estructuras en tiempo de lectura?

La capa de lógica del microservicio lee el campo (schema version) y ejecuta condicionales o fabricas polimorficas para adaptar o rellenar dinámicamente los campos faltantes según la version leida.

19. En un sistema de blogs de alta lectura, un post puede recibir millones de comentarios. Si decidimos embeber todos los comentarios dentro del documento del post, ¿qué riesgo técnico directo identificamos según el 'Subset Pattern'?

Superar el tamaño limite físico del documento JSON/BSON del motor o saturar el buffer de cache en memoria RAM cargando miles de datos históricos irrelevantes para la vista inicial.

20. El 'Subset Pattern' soluciona el problema de los arreglos con crecimiento desmedido. ¿Cuál es la estrategia de diseño detallada que propone?

Mantener embebido dentro del documento principal únicamente un numero limitado y fijo de los elementos mas recientes o leídos, y trasladar el resto de los registros históricos a una colección externa paginada.

21. Un sistema financiero calcula el saldo consolidado de un cliente sumando millones de transacciones históricas en tiempo de ejecución cada vez que abre la app. Al aplicar 'Computed Pattern', ¿cómo se optimiza esta operación?

Se desplaza el costo computacional al momento de la escritura: cada vez que se genera una transacción, el microservicio calcula el nuevo saldo acumulado y lo almacena directamente en un campo dedicado en el documento del cliente.

22. Para combatir ataques de Denegación de Servicio (DoS) en el Core Bancario de FinCore, el API Gateway implementa Rate Limiting y Circuit Breakers. ¿Qué tipo de tácticas de Disponibilidad representan?

Tácticas de recuperación y prevención de fallas, al limitar la tasa de entrada de peticiones externas masivas y aislar proactivamente servicios lentos para proteger los hilos del sistema. *impedir que el tráfico desmedido consuma el pool de hilos previene el desfallecimiento general del host y aísla los componentes degradados*

1 respuestas de simulacro examen

Parte I: Configuración, Infrapatrones y Comunicación

Pregunta 1: ¿Cuál es el impacto principal de sobrecargar un *Microservice Chassis* con excesivas responsabilidades transversales o dependencias pesadas en una arquitectura distribuida?

Respuesta: La respuesta correcta es la **B**. El patrón *Microservice Chassis* busca centralizar y estandarizar las responsabilidades transversales de la arquitectura. Sin embargo, cuando se sobrecarga el componente con excesivas dependencias, frameworks de acoplamiento pesado o mecanismos automatizados redundantes, se penalizan directamente los atributos de desempeño e infraestructura. Esto genera una duplicación en los tiempos de inicialización (*startup time*) y un incremento desproporcionado en el consumo base de memoria RAM de cada contenedor de la malla, afectando la eficiencia de los despliegues.

Pregunta 2: Explique cómo el patrón de *Externalized Configuration* protege los atributos de calidad de seguridad y modificabilidad en entornos multietapa (Dev, QA, Prod).

Respuesta: La respuesta correcta es la **C**. Al implementar *Externalized Configuration*, las variables y propiedades específicas del entorno se inyectan

estrictamente en tiempo de ejecución durante la fase de arranque (*bootstrap*). Esta separación de conceptos protege prioritariamente el atributo de calidad de seguridad, al evitar que credenciales críticas, llaves criptográficas y cadenas de conexión queden expuestas directamente en el código fuente o en los repositorios de control de versiones, facilitando además la modificabilidad al no requerir recompilaciones del artefacto entre entornos.

Pregunta 3: En un sistema de mensajería con garantía de entrega *At-least-once*, ¿cómo mitiga el patrón *Idempotent Consumer* el riesgo de procesar un mensaje duplicado tras un fallo de red?

Respuesta: Bajo una garantía de entrega *At-least-once*, el reenvío de un mensaje duplicado por parte del broker debido a un fallo de red es un escenario esperado. El patrón *Idempotent Consumer* mitiga esto requiriendo que el servicio destino extraiga el identificador único global del mensaje entrante y realice una verificación atómica en una persistencia local o caché de alta velocidad como Redis. Si el identificador ya existe marcado como procesado, el consumidor descarta inmediatamente el *payload* sin alterar el estado del balance de la cuenta, y procede a retornar de manera segura la confirmación (*ACK*) al broker para limpiar la cola.

Pregunta 4: ¿Cuál es la principal desventaja o *trade-off* en términos de desempeño al implementar el patrón *Idempotent Consumer*?

Respuesta: La respuesta correcta es la **B**. La principal desventaja del patrón *Idempotent Consumer* en entornos de procesamiento masivo es el impacto inevitable en la latencia. Debido a que cada evento requiere obligatoriamente una operación síncrona de lectura y posterior escritura en una base de datos o almacenamiento de control para validar la duplicidad, se introduce contención y un cuello de botella de rendimiento que limita la tasa máxima de operaciones por segundo del consumidor.

Pregunta 5: Analice el impacto en los atributos de calidad de disponibilidad y desempeño al sustituir llamadas síncronas (RPI) por *Domain Events*.

Respuesta: La respuesta correcta es la **B**. El uso de *Domain Events* eleva sustancialmente la disponibilidad del microservicio emisor, ya que este se desvincula por completo de los servicios consumidores, liberando sus hilos inmediatamente tras publicar el mensaje en el broker distribuido. No obstante, el procesamiento total del flujo se vuelve eventual y asíncrono, lo que degrada el desempeño de la transacción global en términos de predictibilidad y tiempo de respuesta inmediato para el cliente.

Pregunta 6: Para un pipeline analítico pesado donde el cliente no requiere una respuesta inmediata, ¿por qué el patrón de *Messaging* favorece la disponibilidad del sistema ante la caída del receptor?

Respuesta: Para un pipeline analítico pesado donde el cliente no requiere una respuesta síncrona, la decisión de diseño óptima es implementar el patrón de *Messaging*. Desde la óptica de la disponibilidad, si el microservicio receptor encargado de procesar los reportes sufre una caída o degradación, el broker retiene y persiste los eventos de forma segura en sus colas; una vez el servicio se recupera, reanuda el procesamiento a su propio ritmo sin alterar la operación ni lanzar errores de *timeout* hacia el servicio emisor.

Pregunta 7: ¿Qué impacto genera el antipatrón de *Fat Events* (Eventos Gordos) en la modificabilidad y el acoplamiento entre microservicios?

Respuesta: El antipatrón de *Fat Events* introduce un acoplamiento estructural severo entre los contextos delimitados (*Bounded Contexts*). Al incluir todo el modelo de datos de la entidad en el evento, cualquier cambio menor en el esquema interno del emisor obliga a modificar, compilar y redespargar en cascada a todos los microservicios consumidores de la red, destruyendo los atributos de mantenibilidad, modificabilidad y autonomía de los equipos de desarrollo.

Pregunta 8: Si el servidor centralizado de configuración (v.g., Spring Cloud Config Server) sufre una caída, ¿cómo se ven afectadas las instancias de microservicios que ya están corriendo en producción?

Respuesta: La respuesta correcta es la **B**. Debido a que los microservicios cargan y guardan en su memoria RAM local las propiedades configuradas al momento de su inicialización, la caída del servidor central de configuración no interrumpe el procesamiento de la lógica de negocio activa en producción. Sin embargo, la disponibilidad se ve amenazada a nivel operativo, dado que el sistema no podrá procesar recargas dinámicas de propiedades ni aprovisionar con éxito nuevas instancias horizontales ante picos de tráfico.

Pregunta 9: Explique cómo el *Microservice Chassis* ayuda a mitigar la acumulación de deuda técnica en un ecosistema con decenas de microservicios independientes.

Respuesta: El *Microservice Chassis* actúa como una restricción arquitectónica indispensable para el control de la deuda técnica distribuida. Al encapsular las políticas corporativas de recolección de logs, telemetría, seguridad mTLS y auditoría en una única librería o plantilla base compartida, se garantiza que las actualizaciones de infraestructura críticas se propaguen

de forma homogénea mediante el control de versiones, impidiendo que los equipos implementen soluciones manuales y heterogéneas.

Pregunta 10: Complete la afirmación: El uso del patrón *Idempotent Consumer* requiere obligatoriamente que cada mensaje emitido por el sistema posea un...

Respuesta: El uso del patrón *Idempotent Consumer* requiere obligatoriamente que cada mensaje emitido por el sistema posea un **identificador único global** generado en el origen.

Parte II: Estrategias de Despliegue e Infraestructura

Pregunta 11: ¿Por qué el patrón de *Multiple Service Instances per Host* degrada el aislamiento y pone en riesgo la disponibilidad de servicios críticos coexistentes?

Respuesta: La respuesta correcta es la **B**. El patrón de múltiples instancias por host destruye el aislamiento y confinamiento de recursos a nivel de infraestructura. Si un proceso pesado y demandante como la generación analítica de reportes compite agresivamente por la CPU y memoria RAM del servidor compartido, despojará de hilos de procesamiento al microservicio de autenticación, provocando degradación masiva de latencia o su caída total (*Crash*), afectando la disponibilidad.

Pregunta 12: Desde la perspectiva de la infraestructura, ¿cuál es la razón técnica por la que el escalamiento horizontal basado en contenedores es exponencialmente más rápido que en máquinas virtuales tradicionales?

Respuesta: La respuesta correcta es la **C**. Los contenedores ofrecen un escalamiento horizontal exponencial en comparación con las máquinas virtuales tradicionales debido a que comparten de forma nativa el kernel del sistema operativo del host anfitrión. Al no tener que simular hardware virtual ni cargar con el peso y la inicialización de un sistema operativo invitado completo, el tamaño de las imágenes se reduce drásticamente y los tiempos de arranque pasan de minutos a milisegundos.

Pregunta 13: Identifique el principal riesgo de seguridad asociado al patrón *Service Instance per Container* derivado de compartir recursos del sistema operativo anfitrión.

Respuesta: La respuesta correcta es la **B**. Desde la perspectiva de seguridad, el despliegue basado en contenedores presenta una superficie de ataque compartida más vulnerable, ya que el aislamiento se realiza por software

mediante *namespaces* y *cgroups* sobre el mismo kernel de Linux. Un exploit de escape de contenedor exitoso compromete directamente el núcleo del sistema operativo del host, otorgando al atacante acceso potencial al resto de contenedores e infraestructura subyacente que residan en esa máquina.

Pregunta 14: ¿Qué causa el fenómeno de *Cold Start* (Arranque en Frío) en arquitecturas *Serverless* y qué atributo de calidad penaliza severamente?

Respuesta: El fenómeno de *Cold Start* en arquitecturas *Serverless* se produce cuando ingresa una petición dirigida a una función que se encuentra inactiva o apagada por falta de uso. Al no existir un entorno de ejecución persistido en caliente, el proveedor de nube debe aprovisionar la infraestructura subyacente de forma inmediata, levantar el micro-contenedor, inicializar el runtime del lenguaje y cargar el código de negocio, inyectando una latencia severa que afecta negativamente al desempeño del primer usuario.

Pregunta 15: Evalúe el impacto de centralizar todas las funciones de enrutamiento y seguridad en un único *API Gateway* masivo respecto al riesgo de disponibilidad.

Respuesta: La respuesta correcta es la **B**. Aunque el *API Gateway* provee excelentes capacidades de seguridad y enrutamiento, centralizar todo el tráfico entrante de los clientes externos en una única compuerta tecnológica introduce un Punto Único de Falla (*Single Point of Failure - SPOF*). Si este componente sufre una degradación crítica o una falla completa de infraestructura, todo el ecosistema de microservicios transaccionales queda aislado e inaccesible para el mundo exterior.

Pregunta 16: ¿Cómo aborda el patrón *Backends for Frontends (BFF)* las limitaciones de un *API Gateway* monolítico para mitigar la contención de red?

Respuesta: La respuesta correcta es la **B**. Para mitigar el riesgo asociado a un gateway masivo monolítico, se implementa el patrón de *Backends for Frontends (BFF)*. Este enfoque segrega la puerta de entrada única en múltiples gateways especializados y acoplados a los requerimientos específicos de cada tipo de interfaz de usuario, distribuyendo la carga de red, disminuyendo la contención de hilos de ejecución y aislando los fallos de manera modular.

Pregunta 17: ¿Por qué el uso del patrón *Service Instance per VM* resulta inadecuado en escenarios que exigen una elasticidad inmediata ante picos masivos de tráfico repentinos?

Respuesta: Desplegar instancias basadas en *Service Instance per VM* para un escenario de alta elasticidad como el Black Friday es inviable debido a la latencia en el aprovisionamiento de recursos. Levantar una máquina virtual implica inicializar hipervisores y cargar las abstracciones del sistema operativo invitado, un flujo que toma minutos y que impide reaccionar a tiempo para mitigar ráfagas simultáneas de peticiones masivas en alta concurrencia.

Pregunta 18: Explique el *trade-off* de mantenibilidad operativa frente al riesgo de *Vendor Lock-in* al adoptar el patrón *Serverless Deployment*.

Respuesta: El patrón *Serverless Deployment* optimiza la mantenibilidad operativa al delegar la administración de la infraestructura y el aprovisionamiento físico al proveedor de nube. Sin embargo, deteriora la mantenibilidad y evolución a largo plazo del código de negocio debido al fenómeno de *Vendor Lock-in*, ya que las funciones atómicas se desarrollan acoplándose fuertemente a los SDKs, sistemas de eventos y APIs propietarias de la nube elegida.

Pregunta 19: ¿Cuáles son las tres responsabilidades nativas de una plataforma de despliegue (e.g., Kubernetes) destinadas a proteger la disponibilidad de los servicios?

Respuesta: Una plataforma de despliegue de servicios (como Kubernetes) resuelve nativamente tres aspectos para proteger la disponibilidad: primero, el balanceo de carga automatizado para distribuir equitativamente el tráfico; segundo, el escalamiento horizontal dinámico basado en métricas de consumo de hardware; y tercero, los mecanismos de auto-reparación (*Self-healing*), que monitorean la salud de los contenedores para destruirlos y recrearlos de inmediato si fallan.

Pregunta 20: Detalle cómo el mecanismo de *cgroups* del kernel de Linux previene que un microservicio corrupto degrade a otras instancias en el mismo host.

Respuesta: El aislamiento de recursos se garantiza mediante el uso de los mecanismos de *cgroups* (grupos de control) administrados por el kernel del sistema operativo host y configurados por el orquestador. Al definir cuotas y límites máximos de memoria RAM por contenedor, el sistema operativo activa el *Out-Of-Memory (OOM) Killer* para finalizar de forma controlada el proceso de la instancia corrupta tan pronto exceda su tope, impidiendo que afecte el rendimiento de los demás componentes del host.

Parte III: Descubrimiento de Servicios (Service Discovery)

Pregunta 21: ¿Cuál es la principal desventaja en términos de acoplamiento de infraestructura y mantenibilidad al delegar el registro en el patrón *Self Registration*?

Respuesta: La respuesta correcta es la **B**. En el esquema de *Self Registration*, la lógica del ciclo de vida se integra en el microservicio, siendo la propia instancia la encargada de llamar al *Service Registry* para notificar su estado activo. Esto introduce un riesgo de modificabilidad y mantenibilidad al acoplar el código de negocio a librerías de infraestructura del registro, complejizando la gestión en ecosistemas políglotas con múltiples pilas tecnológicas.

Pregunta 22: ¿Cómo detecta e inscribe un *3rd Party Registrar* las nuevas instancias en el *Service Registry* de forma agnóstica al código de negocio?

Respuesta: La respuesta correcta es la **B**. El componente *Registrar* opera de manera externa al código del microservicio. Este demonio de infraestructura escucha activamente los flujos de eventos emitidos por el orquestador del clúster o el motor de contenedores (por ejemplo, los eventos de la API de Kubernetes). Al detectar que una nueva instancia ha completado con éxito la fase de inicialización, extrae de forma automática sus metadatos de red y la da de alta en el registro.

Pregunta 23: Ordene cronológicamente el flujo estricto que sigue el patrón *Client-side Discovery* para efectuar una llamada entre un servicio emisor y uno receptor.

Respuesta: La respuesta correcta es la **B**. El flujo cronológico estricto para el descubrimiento del lado del cliente inicia cuando el emisor consulta el *Service Registry* pidiendo la ubicación de red del receptor (2). Posteriormente, el registro retorna un arreglo de direcciones de red válidas de las instancias activas (4). El emisor ejecuta en su propio espacio de memoria un algoritmo de balanceo de carga sobre la lista (3) y finalmente realiza la invocación directa al nodo seleccionado (1).

Pregunta 24: Explique por qué el patrón *Server-side Discovery* simplifica el diseño y favorece la mantenibilidad en ecosistemas microservicios con arquitecturas políglotas.

Respuesta: El patrón de *Server-side Discovery* destaca en mantenibilidad para entornos políglotas debido a que abstrae por completo los detalles de red del cliente. El microservicio emisor no requiere la incorporación de

clientes SDK dedicados, lógicas de balanceo dinámico de hilos ni dependencias escritas en el mismo lenguaje del registro; simplemente emite una petición estandarizada dirigida a un enrutador intermedio, permitiendo la interoperabilidad sin fricciones de cualquier pila tecnológica.

Pregunta 25: Si una instancia de microservicio falla abruptamente bajo un esquema de *Self Registration* con un TTL de expiración de 90 segundos, analice el impacto en el tráfico durante esa ventana.

Respuesta: Durante esa ventana de 90 segundos, los clientes experimentarán fallos continuos de omisión de servicio, errores de conexión (*Connection Refused*) y *timeouts*. Debido a que la instancia murió de forma abrupta sin enviar la señal de baja, el *Service Registry* mantendrá mapeada su ubicación como válida hasta que expire el tiempo de vida (*TTL*). Como resultado, los algoritmos de enrutamiento del lado del cliente seguirán enviando peticiones a un puerto inactivo de red.

Pregunta 26: Detalle la sinergia arquitectónica resultante de combinar los patrones *3rd Party Registration* y *Server-side Discovery* (tal como opera de forma nativa en Kubernetes).

Respuesta: La combinación de ambos patrones libera al desarrollo al automatizar por completo las capas de red dinámicas. Cuando un contenedor es aprovisionado, el *3rd Party Registrar* detecta su existencia a nivel de orquestador y lo inscribe en el mapa de servicios de manera autónoma. Paralelamente, cuando un cliente requiere consumir dicho servicio, dirige la petición a un proxy o balanceador que aplica *Server-side Discovery*; este proxy lee dinámicamente las IPs del registro para desviar el tráfico, eliminando configuraciones manuales de red.

Pregunta 27: ¿Cuál es el beneficio directo en términos de latencia y desempeño al almacenar el mapa del *Service Registry* en una caché local del cliente emisor?

Respuesta: La respuesta correcta es la **B**. Almacenar localmente el mapa de direccionamiento en una caché del lado del cliente disminuye drásticamente la latencia acumulada en las peticiones subsiguientes. Esto se debe a que el servicio emisor evita ejecutar una llamada adicional a través de la red hacia el *Service Registry* antes de cada transacción, permitiéndole enrutar las peticiones punto a punto con el nodo destino de manera inmediata.

Pregunta 28: Ante una partición de red en el clúster de infraestructura, analice el dilema de diseño del *Service Registry* bajo los postulados del Teo-

rema de CAP.

Respuesta: De acuerdo al Teorema de CAP, ante una partición de red, el arquitecto debe elegir entre consistencia y disponibilidad. Si se prioriza la consistencia (CP), el registro rechazará o bloqueará altas y modificaciones dudosas de IPs para mitigar el riesgo de datos corruptos, impactando la disponibilidad del descubrimiento. Si se prefiere disponibilidad (AP), el sistema continuará respondiendo consultas con el mapa local actual, aceptando un modelo de consistencia eventual a riesgo de retornar temporalmente ubicaciones de red desactualizadas o caídas.

Pregunta 29: Explique por qué un endpoint de *Health Check* robusto debe evaluar dependencias lógicas internas y no limitarse a validar la conectividad de red básica.

Respuesta: La función crítica de un *Health Check* nativo en el *Service Registry* es validar de manera empírica y periódica la viabilidad operativa real de una instancia, y no solo su conectividad básica de red. El registro invoca un endpoint dedicado expuesto por el microservicio para verificar que sus dependencias internas, conexiones a bases de datos y pools de hilos se encuentren estables; ante cualquier anomalía recurrente, remueve automáticamente la IP del pool para proteger la integridad del tráfico global.

Pregunta 30: Para integrar un módulo legacy inalterable (v.g., COBOL) con microservicios dinámicos modernos, ¿cuál es la combinación de patrones de descubrimiento obligatoria?

Respuesta: Para un ecosistema heredado compuesto por COBOL y tecnologías modernas, la elección arquitectónica obligatoria es el uso conjunto de *3rd Party Registration* y *Server-side Discovery*. Dado que es inviable modificar el código fuente del monolito de COBOL para inyectar capacidades de descubrimiento, un agente externo se encarga de censar su infraestructura fija y mantener actualizado el registro. Al mismo tiempo, los microservicios modernos consumen las operaciones llamando a un proxy centralizado, abstractando las complejidades de infraestructura de ambos extremos.

Parte IV: Gestión de Persistencia y Patrones de Datos

Pregunta 31: Al implementar *Database per Service*, ¿cuál es la restricción técnica más severa que imposibilita las consultas agregadas estructuradas tradicionales?

Respuesta: La respuesta correcta es la **B**. El aislamiento estricto impuesto por el patrón *Database per Service* elimina el esquema relacional monolítico

unificado, forzando a que las entidades de dominio residan en motores físicos independientes. Esto impide de forma categórica la ejecución de sentencias SQL estructuradas basadas en la cláusula `JOIN` en una sola base de datos, obligando a los desarrolladores a recurrir a técnicas de composición en la capa de aplicación o sincronización asíncrona de modelos.

Pregunta 32: Señale el principal *trade-off* negativo del patrón *API Composition* en flujos con múltiples llamadas secuenciales síncronas respecto a la latencia.

Respuesta: La respuesta correcta es la **B**. El patrón *API Composition* opera bajo un esquema de orquestación puramente síncrono. La latencia total percibida por el cliente equivale directamente al tiempo de respuesta del microservicio más lento dentro de la cadena distribuidora; además, al carecer de un modelo de persistencia optimizado para consultas agregadas, la caída o degradación de un solo servicio crítico del pipeline invalida el procesamiento completo o corrompe la vista devuelta al frontend.

Pregunta 33: Explique el flujo de sincronización asíncrona entre el *Write Side* y el *Read Side* en un sistema de alta concurrencia guiado por CQRS.

Respuesta: En una arquitectura CQRS, el flujo se inicia cuando el cliente envía un comando de modificación al *Write Side*, el cual valida las reglas del dominio y persiste los cambios de forma normalizada en la base de datos de escritura. Posterior a la confirmación local, este componente emite de forma asíncrona un evento de dominio hacia el broker de mensajería. Un proyector indexador suscrito al broker consume el evento, extrae los campos mutados y actualiza de manera desnormalizada la base de datos del *Read Side* (por ejemplo, un motor de Elasticsearch), quedando disponible para las consultas bajo un esquema de consistencia eventual.

Pregunta 34: Describa el mecanismo de rehidratación en *Event Sourcing* y detalle el propósito del patrón complementario de *Snapshots*.

Respuesta: En *Event Sourcing*, para conocer el estado actual de un objeto, la aplicación debe ejecutar un proceso de rehidratación, recuperando de forma cronológica e inmutable la totalidad de los eventos del *Event Store* para reproducirlos secuencialmente en memoria. Para optimizar el desempeño y mitigar la latencia provocada por procesar millones de eventos históricos, se aplica complementariamente el patrón de *Snapshots* (Instantáneas); este guarda de forma persistentemente el estado consolidado de la entidad cada determinado número de eventos, permitiendo que la rehidratación inicie desde la instantánea más reciente.

Pregunta 35: Si dos servicios independientes comparten una única *Shared Database*, analice cómo una transacción larga ACID de uno degrada el desempeño del otro.

Respuesta: Al compartir una misma base de datos, el bloqueo prolongado de filas o tablas impuesto por las transacciones ACID locales largas del servicio de cobros genera contención inmediata sobre los recursos físicos del motor de base de datos (bloques de disco, conexiones del pool, memoria caché y CPU). Esta fricción degrada masivamente el desempeño del servicio de envíos independiente que intenta leer o escribir sobre las mismas tablas, comprometiendo severamente su disponibilidad debido a un acoplamiento a nivel de persistencia de datos.

Pregunta 36: ¿Por qué el patrón CQRS exhibe una baja facilidad de mantenimiento operativa en comparación con las arquitecturas monolíticas CRUD tradicionales?

Respuesta: El patrón CQRS es calificado con una baja facilidad de mantenimiento debido a dos retos de ingeniería: en primer lugar, duplica la complejidad del desarrollo al obligar a los equipos a diseñar, codificar y evolucionar dos modelos de datos completamente independientes (el relacional/normalizado para comandos y el desnormalizado optimizado para consultas). En segundo lugar, introduce la carga operativa de gestionar y monitorear la infraestructura distribuidora de mensajería encargada de la sincronización asíncrona, requiriendo el desarrollo de lógicas adicionales para manejar los desfases inherentes a la consistencia eventual.

Pregunta 37: Explique cómo gestiona el patrón *Saga* la consistencia de datos distribuida mediante el uso exclusivo de transacciones compensatorias locales.

Respuesta: La respuesta correcta es la **B**. A diferencia de los protocolos tradicionales de confirmación en dos fases (*2PC*), el patrón *Saga* no bloquea de manera global los recursos de las bases de datos distribuidas. Si se produce un fallo en un paso intermedio de la secuencia, el orquestador o la cadena de eventos dispara intencionalmente transacciones compensatorias diseñadas explícitamente en los microservicios previos para deshacer o neutralizar lógicamente los efectos de las transacciones locales ya confirmadas, restableciendo la consistencia del sistema de manera distribuida.

Pregunta 38: Defina formalmente el rol y las restricciones operativas inmutables de un almacén de datos tipo *Event Store*.

Respuesta: La respuesta correcta es la **C**. El *Event Store* constituye la

base de datos especializada y de tipo *append-only* encargada de registrar de manera inmutable, cronológica y altamente indexada las corrientes de eventos de dominio, representando el único origen de la verdad dentro de arquitecturas fundamentadas en *Event Sourcing*.

Pregunta 39: ¿Qué implicaciones tiene la consistencia eventual en interfaces de usuario distribuidas y qué táctica de Frontend enmascara dicho desfase de tiempo?

Respuesta: La consistencia eventual establece que, en sistemas distribuidos desacoplados, los modelos de lectura pueden experimentar un breve desfase de tiempo antes de reflejar las mutaciones confirmadas en el almacenamiento de escritura. Para evitar degradar la experiencia de usuario (*UI/UX*), el frontend debe implementar técnicas de actualización optimista (*Optimistic UI Updates*), asumiendo un procesamiento exitoso inmediato en la interfaz visual, o incorporar animaciones de transición controladas que enmascaren el tiempo de sincronización asíncrona subyacente del backend.

Pregunta 40: Explique el objetivo del patrón *Command-side Replica* para mitigar la saturación durante la validación estricta de invariantes de negocio.

Respuesta: El patrón *Command-side Replica* busca mitigar la contención de lectura y la saturación del almacenamiento transaccional principal durante flujos de alta concurrencia de escrituras. Este enfoque mantiene una réplica altamente optimizada y local para cubrir necesidades de consulta específicas requeridas estrictamente por el mismo contexto de validación del comando, priorizando de manera agresiva los atributos de desempeño e integridad transaccional inmediata en la toma de decisiones del dominio.

Parte V: Diseño Detallado de Persistencia Documental

Pregunta 41: ¿Cómo permite el patrón *Schema Versioning* la evolución del modelo documental NoSQL sin requerir ventanas de mantenimiento globales para migraciones masivas?

Respuesta: La respuesta correcta es la **B**. El patrón detallado *Schema Versioning* requiere inyectar un campo numérico con la versión del esquema dentro de cada documento JSON individual. Esto permite que coexistan registros con estructuras heterogéneas dentro de la misma colección de producción; la aplicación interpreta polimórficamente los datos basándose en el número de versión leído, habilitando despliegues continuos y modificaciones modulares en el modelo de persistencia sin necesidad de realizar migraciones

monolíticas globales en ventanas de mantenimiento.

Pregunta 42: Detalle la reestructuración JSON impuesta por el *Attribute Pattern* y justifique su impacto positivo en el consumo de memoria RAM del motor de base de datos.

Respuesta: La reestructuración del JSON aplicando el *Attribute Pattern* desplaza los campos variables hacia un arreglo de subdocumentos con una estructura llave-valor uniforme: `{ "atributos": [ { "k": "color", "v": "Azul" }, { "k": "talla", "v": "42" } ] }`. Esta decisión optimiza radicalmente el desempeño debido a que permite al motor NoSQL indexar la totalidad de las características dinámicas utilizando un único índice compuesto multiclave sobre el arreglo (`"atributos.k": 1, "atributos.v": 1`), evitando la creación y el mantenimiento de decenas de árboles B-Tree independientes que saturarían la memoria RAM del servidor.

Pregunta 43: Analice cómo el *Subset Pattern* protege la memoria de trabajo (*Working Set*) controlando el crecimiento indefinido de documentos (efecto *unbounded grow*).

Respuesta: El *Subset Pattern* aborda el problema dividiendo los datos basados en su frecuencia y volumen de acceso. En lugar de permitir que un documento crezca de forma indefinida degradando la memoria de trabajo (*Working Set*) del motor NoSQL, este patrón mantiene embebido dentro del documento principal únicamente un subconjunto acotado y ordenado con los registros más recientes o consultados (por ejemplo, los últimos 50 comentarios). La masa gigante de datos históricos remanentes se desplaza hacia una colección externa altamente indexada, manteniendo el documento raíz ligero y optimizando las lecturas del feed.

Pregunta 44: Describa el mecanismo del *Computed Pattern* para optimizar las lecturas complejas de agregación, detallando el costo computacional en la fase de escritura.

Respuesta: La respuesta correcta es la **B**. Bajo el *Computed Pattern*, el costo del cálculo matemático se desplaza deliberadamente hacia la fase de escritura. Cada vez que se confirma una nueva transacción de compra, la aplicación ejecuta una operación atómica de incremento sobre un campo precalculado persistido denominado `gasto_total_mes` dentro del documento del usuario. Cuando el dashboard solicita la lectura, el dato se retorna inmediatamente de forma directa con una complejidad temporal de orden $\mathcal{O}(1)$, evitando ejecuciones costosas de agregación sobre el motor en tiempo de ejecución.

Pregunta 45: Identifique el patrón NoSQL orientado a mitigar el impacto en el rendimiento provocado por escrituras masivas sobre contadores analíticos guardando aproximaciones en memoria.

Respuesta: El patrón de diseño NoSQL orientado a mitigar el impacto en el rendimiento provocado por escrituras concurrentes masivas sobre contadores analíticos, guardando aproximaciones agregadas temporalmente en memoria antes de impactar el disco, se denomina **Approximation Pattern**.

Parte VI: Seguridad Avanzada (Caso FinCore - STRIDE y PACE)

Pregunta 46: En una red interna bancaria, ¿a qué categoría STRIDE pertenece la manipulación de tramas en tránsito y qué táctica de cifrado simétrico la neutraliza?

Respuesta: La respuesta correcta es la **B**. La alteración maliciosa o manipulación no autorizada de tramas de datos en tránsito a través de la red interna corresponde de forma estricta a la categoría de **Tampering** dentro del modelo de amenazas STRIDE. La táctica de diseño arquitectónico idónea para neutralizar este riesgo consiste en blindar los canales de comunicación de la malla de servicios implementando **mTLS (Mutual TLS)** acoplado a suites de cifrado fuertes, garantizando la autenticidad simétrica de los extremos y la integridad criptográfica inalterable de los payloads.

Pregunta 47: Analice la amenaza de *Elevation of Privilege* en el contexto de tokens manipulados en el cliente y enuncie el principio básico de validación Zero Trust.

Respuesta: La respuesta correcta es la **B**. Cuando un atacante altera claims o burla los controles de la interfaz visual para ejecutar transacciones reservadas a roles administrativos, se materializa la amenaza de **Elevation of Privilege**. El principio de mitigación correcto exige implementar validaciones estrictas y centralizadas de control de acceso basado en roles o atributos (RBAC/ABAC) en el backend de cada microservicio transaccional independiente; bajo el axioma de Zero Trust, nunca se debe delegar la confianza del estado seguro a las validaciones de las aplicaciones del cliente.

Pregunta 48: Explique el costo computacional y la penalización de latencia sobre el desempeño que impone el cumplimiento estricto del estilo de seguridad arquitectónico PACE.

Respuesta: El cumplimiento estricto del estilo de seguridad arquitectónico PACE (Perímetro, Acceso, Cifrado, Auditoría) impone una penalización ma-

siva sobre el atributo de calidad de **desempeño**, manifestándose como un incremento severo en la latencia global del sistema. Este detrimento es el resultado directo del costo computacional acumulado por los procesos recurrentes de cifrado y descifrado a nivel de campo (FLE), el apretón de manos criptográfico de mTLS en cada salto de infraestructura y el procesamiento de firmas inmutables de los tokens de acceso y las pistas de auditoría bancaria.

Pregunta 49: Ante la amenaza de *Repudiation*, ¿cómo blindas la arquitectura de FinCore las pistas de auditoría integrando almacenamiento WORM y el estándar RFC 3161?

Respuesta: La situación descrita mapea de forma directa la categoría de **Repudiation (Repudio)** dentro del modelo STRIDE, en la cual un actor niega haber sido el autor de una transacción de negocio. La arquitectura de seguridad de FinCore blindas legalmente a la institución financiera mediante la convergencia de dos tecnologías: primero, el almacenamiento inmutable de las pistas de auditoría integral en arreglos de discos con tecnología **WORM** (*Write Once, Read Many*); y segundo, el sellado de tiempo criptográfico inalterable bajo el estándar **RFC 3161** emitido por una Autoridad de Sellado de Tiempo (*TSA*) externa, certificando matemáticamente la fecha, hora exacta y autoría inalterable del evento.

Pregunta 50: Explique el mecanismo criptográfico descentralizado que faculta a los *Access Tokens* basados en JWT para validar identidades locales eliminando saltos de red adicionales al IdP.

Respuesta: La respuesta correcta es la **B**. A diferencia de los mecanismos tradicionales basados en cookies de sesión orientadas a bases de datos relacionales centrales, los tokens *JWT* son artefactos auto-contenidos y firmados criptográficamente. Esto faculta a cada microservicio independiente a validar de forma matemática, local y descentralizada la identidad, los roles y la expiración de la sesión del usuario descifrando la firma con la llave pública del emisor, eliminando saltos de red recurrentes hacia el Identity Provider (*IdP*) y maximizando la escalabilidad del sistema distribuido.